Table des matières

Labo 04 — Bâtir l'étoile des ventes 1
Avant de commencer 2
Étape 1 — Installer `dbt_utils` 2
Étape 2 — La dernière vue de staging 3
Étape 3 — Deux couches de plus dans le projet 5
Étape 4 — `dim_date`, fabriquée par `date_spine` 6
Étape 5 — `dim_concessionnaire` et sa clé 7
Étape 6 — `dim_produit` 8
Étape 7 — Le modèle qui n'existera nulle part 9
Étape 8 — `fct_ventes` 10
Étape 9 — Le grain, puis la réconciliation 11
Étape 10 — Déclarer ce qu'on livre 13
Récapitulatif 14
Dépannage 15

Labo 04 — Bâtir l'étoile des ventes

Labo · Cours 4/9 — Formation Analytics Engineering

Ce que vous allez faire. Installer le seul paquet de la formation, ouvrir les deux couches qui manquent au projet, et bâtir l'étoile des ventes : une dimension de dates fabriquée de toutes pièces, deux dimensions tirées du staging, un modèle intermédiaire qui n'existera nulle part, et le fait des lignes de commande. Vous finirez par le contrôle qui décide si l'étoile se livre ou non : sa valeur nette doit égaler au cent près celle que l'ERP porte dans ses en-têtes. Objectif de fin : 75 652 lignes de fait, 430 148 012,16 $, 0,00 $ d'écart.

Deux terminaux servent dans ce labo. L'Anaconda Prompt, environnement `formations` actif, sert à dbt et à Git ; il est ouvert dans `Projet_Boreal/02_ae`. La ligne de commande DuckDB sert à vérifier ce que dbt a écrit, et elle doit être fermée quand dbt travaille.

La liste de contrôle du cours :

- ☐ `dbt_utils` installé, et `dbt_packages/` hors du dépôt

© 2026 iAlexMG. Tous droits réservés.

- ☐ `erp_vbap` déclarée en source, et `stg_postes` : 77 732 postes, dont 2 080 refusés
- ☐ Les couches `intermediate` et `marts` déclarées dans `dbt_project.yml`
- ☐ `dim_date` : 1 551 jours, du 2022-01-01 au 2026-03-31, libellés en français
- ☐ `dim_concessionnaire` : 156 lignes, 156 clés
- ☐ `dim_produit` : 68 articles
- ☐ `int_lignes_enrichies` absent de la base, présent dans le graphe
- ☐ `fct_ventes` : 75 652 lignes, autant de couples distincts
- ☐ La réconciliation avec l'ERP : 0,00 $ d'écart
- ☐ La première section de `LIVRAISON.md` écrite

Le labo se fait en une séance. Les Cours 02 et 03 en demandaient deux ; celui-ci se fait d'un trait.

Avant de commencer

Ce labo part du projet tel que le Cours 03 l'a laissé. Vérifiez-le avant d'ajouter quoi que ce soit, dans l'Anaconda Prompt, depuis `Projet_Boreal/02_ae` :

```
dbt debug
dbt ls
```

`dbt debug` doit rendre « All checks passed! ». `dbt ls` doit lister quatre modèles, les trois vues de staging du labo précédent et le modèle de contrôle, plus trois sources préfixées de `source:`. Si `stg_articles` manque, c'est la question P1 des exercices du Cours 03 : écrivez-la avant de continuer, l'étoile en a besoin, et sa source `erp_mara` avec elle.

Fermez ensuite la ligne de commande DuckDB si elle est ouverte, par `.exit` : un seul processus à la fois sur l'entrepôt.

Étape 1 — Installer `dbt_utils`

Théorie : `Theorie_04_AE.md`, section 2.

Créez `packages.yml` à la racine du projet, à côté de `dbt_project.yml` :

© 2026 iAlexMG. Tous droits réservés.

```yaml
packages:
  - package: dbt-labs/dbt_utils
    version: [">=1.0.0", "<2.0.0"]
```

La fourchette dit ce que le projet accepte : toute version 1 du paquet, aucune version 2. Une version majeure change les signatures des macros, et personne ne veut que cela arrive pendant une nuit de chargement.

```
dbt deps
```

Attendu, trois lignes qui comptent :

```
Updating lock file in file path: ...\02_ae/package-lock.yml
Installing dbt-labs/dbt_utils
Installed from version 1.4.1
```

Deux fichiers sont apparus. `package-lock.yml` porte la version exactement résolue, et il se versionne : c'est lui qui garantit que votre collègue installera la même. Le dossier `dbt_packages/`, lui, porte le code téléchargé et ne se versionne pas ; il est déjà dans le `.gitignore` que vous avez écrit au Cours 01, avec sa raison.

Vérifiez que Git est du même avis :

```
git status --short
```

`packages.yml` et `package-lock.yml` doivent apparaître comme des fichiers neufs, et `dbt_packages/` ne doit pas apparaître du tout.

Étape 2 — La dernière vue de staging

Théorie : `Theorie_03_AE.md`, sections 4 et 6.

Le fait des ventes lit les postes de commande, et le Cours 03 n'a pas écrit leur vue. C'est son geste, appliqué une dernière fois : déclarer la source, puis renommer, typer, décoder, tirer l'état courant.

La source d'abord, parce que dbt refuse de deviner. Ajoutez la table sous `tables:` dans `models/staging/_sources.yml` :

```yaml
      - name: erp_vbap
        description: "Les postes de commande."
```

C'est la troisième fois que vous faites ce geste, et c'est voulu : une source se déclare le jour où un modèle la lit, pas avant. Si vous sautez l'ajout, la commande suivante s'arrête sur `Compilation Error`, avec `Model 'model.boreal_ae.stg_postes' ... depends on a source named 'raw.erp_vbap' which was not found`.

Dans `models/staging/stg_postes.sql` :

© 2026 iAlexMG. Tous droits réservés.

```sql
with courant as (

    select
        *,
        row_number() over (
            partition by vbeln, posnr
            order by _lot desc
        ) as rang

    from {{ source('raw', 'erp_vbap') }}

)

select
    vbeln as numero_commande,
    posnr as numero_poste,
    matnr as code_article,
    nullif(abgru, '') as motif_refus,
    kwmeng::decimal(18, 3) as quantite,
    netwr::decimal(18, 2) as valeur_nette,
    wavwr::decimal(18, 2) as cout,
    waerk as devise,
    _lot as lot_source

from courant
where rang = 1
```

Remarquez le `nullif` sur `abgru` : un poste actif porte une chaîne vide dans l'ERP, et une absence se dit mieux par `NULL` que par du vide. Le staging ne décide pas ce qu'on fait des postes refusés, il se contente de les rendre reconnaissables.

```
dbt run --select stg_postes
```

Puis, dans la ligne de commande DuckDB ouverte depuis `Projet_Boreal` :

```sql
select
    count(*) as postes,
    count(*) filter (where motif_refus is not null) as refuses,
    sum(valeur_nette) as valeur_totale,
    sum(valeur_nette) filter (where motif_refus is not null) as valeur_refusee
from staging.stg_postes;

select motif_refus, count(*) as postes, sum(valeur_nette) as valeur
from staging.stg_postes
```


© 2026 iAlexMG. Tous droits réservés.

```
group by 1
order by 1;
```

Mesure	Attendu
Postes	77 732
Dont refusés	2 080
Valeur totale, refusés compris	448 149 563,37 $
Valeur des postes refusés	18 001 551,21 $
`Z1`, annulé par le concessionnaire	1 171 postes, 13 334 225,08 $
`Z2`, délai refusé	121 postes, 3 977 323,13 $
`Z3`, rupture chez le fournisseur	788 postes, 690 003,00 $

Retenez les 448 149 563,37 $: c'est le chiffre que vous obtiendrez à l'étape 9 si vous oubliez d'écarter les postes refusés, et il est parfaitement crédible.

Étape 3 — Deux couches de plus dans le projet

Théorie : `Theorie_04_AE.md`, sections 1 et 9.

```
mkdir models\intermediate
mkdir models\marts
```

Puis, dans `dbt_project.yml`, complétez le bloc `models:` :

```
models:
  boreal_ae:
    staging:
      +materialized: view
      +schema: staging
    intermediate:
      +materialized: ephemeral
    marts:
      +materialized: table
      +schema: marts
```

Trois lignes méritent un mot.

© 2026 iAlexMG. Tous droits réservés.

`+materialized: table` sur `marts` : les tables des `MARTS` sont lues souvent et par plusieurs, donc elles se calculent une fois.

`+schema: marts` : grâce à la macro `generate_schema_name` du Cours 03, le nom sera nu, `marts` et non `staging_marts`.

Et l'absence de `+schema` sur `intermediate`. Ce n'est pas un oubli : un modèle `ephemeral` ne crée aucun objet, donc il n'atterrit dans aucun schéma. Déclarer un schéma pour lui ferait croire le contraire à qui lit le fichier.

Étape 4 — `dim_date`, fabriquée par `date_spine`

Théorie : `Theorie_04_AE.md`, section 4.

Dans `models/marts/dim_date.sql` :

```
with jours as (

    {{ dbt_utils.date_spine(
        datepart="day",
        start_date="cast('2022-01-01' as date)",
        end_date="cast('2026-04-01' as date)"
    ) }}

)

select
    date_day::date as date_jour,
    year(date_day) as annee,
    quarter(date_day) as trimestre,
    month(date_day) as mois,
    ['janvier', 'février', 'mars', 'avril', 'mai', 'juin',
     'juillet', 'août', 'septembre', 'octobre', 'novembre',
     'décembre'][month(date_day)] as nom_mois,
    ['lundi', 'mardi', 'mercredi', 'jeudi', 'vendredi', 'samedi',
     'dimanche'][isodow(date_day)] as nom_jour,
    isodow(date_day) >= 6 as est_fin_de_semaine

from jours
```

Deux points de vigilance. La colonne que la macro produit s'appelle `date_day`, et c'est elle qu'on renomme. La borne de fin est exclue : `2026-04-01` donne un dernier jour au 31 mars.

```
dbt run --select dim_date
```


© 2026 iAlexMG. Tous droits réservés.

Vérifiez dans DuckDB :

```
select count(*) as jours, min(date_jour) as premier, max(date_jour) as dernier
from marts.dim_date;

select annee, count(*) as jours from marts.dim_date group by 1 order by 1;

select date_jour, nom_mois, nom_jour, est_fin_de_semaine
from marts.dim_date
where date_jour in (date '2022-01-01', date '2024-02-29', date '2025-08-15')
order by 1;

select
    count(*) filter (where nom_mois is null) as mois_nuls,
    count(distinct nom_mois) as mois_distincts,
    count(distinct nom_jour) as jours_distincts
from marts.dim_date;
```

Mesure	Attendu
Jours	1 551, du 2022-01-01 au 2026-03-31
Par année	365, 365, 366, 365, puis 90 pour les trois mois de 2026
Le 2022-01-01	janvier, samedi, fin de semaine
Le 2024-02-29	février, jeudi
Le 2025-08-15	août, vendredi
Libellés nuls, mois distincts, jours distincts	0, 12 et 7

La dernière ligne est le contrôle qui compte. Douze mois et sept jours distincts, aucun nul : la liste indexée couvre toutes les valeurs. Une liste trop courte d'un élément ne planterait pas, elle laisserait des `NULL` un jour sur sept.

Étape 5 — `dim_concessionnaire` et sa clé

Théorie : `Theorie_04_AE.md`, sections 3 et 5.

Dans `models/marts/dim_concessionnaire.sql` :

© 2026 iAlexMG. Tous droits réservés.

```sql
select
    {{ dbt_utils.generate_surrogate_key(['code_concessionnaire']) }} as cle_concessionnaire,
    code_concessionnaire,
    nom_concessionnaire,
    ville,
    province,
    palier,
    date_creation,
    est_supprime

from {{ ref('stg_concessionnaires') }}
```

Le modèle est court, et c'est le signe que le Cours 03 a fait son travail : le décodage du palier et l'état courant sont derrière vous. La dimension n'ajoute qu'une clé.

```
dbt run --select dim_concessionnaire
```

```sql
select
    count(*) as lignes,
    count(distinct cle_concessionnaire) as cles,
    count(*) filter (where palier is null) as paliers_nuls
from marts.dim_concessionnaire;

select code_concessionnaire, cle_concessionnaire
from marts.dim_concessionnaire
order by code_concessionnaire
limit 3;

select md5('0000200001') as md5_a_la_main;
```

Attendu : 156 lignes, 156 clés, 0 palier nul. Et la clé du concessionnaire `0000200001` vaut `bb3a0ec37500394ece9165f87dffe079`, exactement ce que `md5` rend à la main. La macro n'a rien de magique, et c'est une bonne nouvelle : le jour où un chiffre vous surprendra, vous pourrez recalculer une clé vous-même.

Les 156 clés pour 156 lignes sont le contrôle d'unicité. Un jour où il ne tombera pas juste, la cause sera presque toujours la même : une clé fabriquée sur une colonne qui n'est pas unique.

Étape 6 — `dim_produit`

Théorie : `Theorie_04_AE.md`, section 5.

Même geste, sur la vue des articles.

© 2026 iAlexMG. Tous droits réservés.

Dans `models/marts/dim_produit.sql` :

```
select
    {{ dbt_utils.generate_surrogate_key(['code_article']) }} as cle_produit,
    code_article,
    gamme,
    type_article,
    annee_modele,
    unite_de_base

from {{ ref('stg_articles') }}
```

```
dbt run --select dim_produit
```

```
select
    count(*) as lignes,
    count(distinct cle_produit) as cles,
    count(*) filter (where gamme is null) as gammes_nulles
from marts.dim_produit;
```

Attendu : 68 articles, 68 clés, 0 gamme nulle.

Remarquez ce que la dimension ne fait pas : il n'y a pas de table `dim_gamme` à côté, avec quatre lignes. La gamme est une colonne, comme le Cours 02 l'a décidé.

Étape 7 — Le modèle qui n'existera nulle part

Théorie : `Theorie_04_AE.md`, section 6.

La couche `INTERMEDIATE` porte le travail qui prépare le fait : la jointure vers l'en-tête, le filtre des postes refusés, le calcul de la marge.

Dans `models/intermediate/int_lignes_enrichies.sql` :

```
with postes_actifs as (

    select *
    from {{ ref('stg_postes') }}
    where motif_refus is null

),

commandes as (
```


© 2026 iAlexMG. Tous droits réservés.

```
    select *
    from {{ ref('stg_commandes') }}

)

select
    p.numero_commande,
    p.numero_poste,
    c.date_creation as date_commande,
    c.code_concessionnaire,
    p.code_article,
    c.type_document,
    p.quantite,
    p.valeur_nette,
    p.cout,
    p.valeur_nette - p.cout as marge

from postes_actifs as p
inner join commandes as c
    on c.numero_commande = p.numero_commande
```

C'est ici que la règle de métier s'écrit, une fois : le fait des ventes ne compte pas les postes refusés. Et c'est ici que la marge se calcule, parce qu'une soustraction est une mesure additive qui a sa place dans un fait.

Essayez maintenant de le construire :

```
dbt run --select int_lignes_enrichies
```

Il ne se passe rien. Pas d'erreur, pas de `PASS`, juste une ligne « Finished running » et un temps. C'est le comportement normal d'un modèle `ephemeral` : il n'y a rien à créer, donc dbt ne crée rien.

Vérifiez qu'il est quand même dans le projet :

```
dbt ls
```

`boreal_ae.intermediate.int_lignes_enrichies` y figure, entre les dimensions et les vues de staging. Le modèle existe pour dbt, il n'existe pas pour la base.

Étape 8 — `fct_ventes`

Théorie : `Theorie_04_AE.md`, sections 3 et 7.

© 2026 iAlexMG. Tous droits réservés.

Dans `models/marts/fct_ventes.sql` :

```
select
    date_commande as cle_date,
    {{ dbt_utils.generate_surrogate_key(['code_concessionnaire']) }} as cle_concessionnaire,
    {{ dbt_utils.generate_surrogate_key(['code_article']) }} as cle_produit,
    numero_commande,
    numero_poste,
    quantite,
    valeur_nette,
    cout,
    marge

from {{ ref('int_lignes_enrichies') }}
```

Douze lignes, et aucune jointure. Les clés des deux dimensions se recalculent par la même macro que dans les dimensions elles-mêmes, la clé de la date est la date, et les deux numéros restent dans le fait comme dimensions dégénérées.

```
dbt run
```

Cette fois, lancez tout : neuf modèles se construisent, cinq vues et quatre tables, en moins d'une seconde. La dernière ligne doit dire `PASS=9 WARN=0 ERROR=0`.

Avant de vérifier les chiffres, allez voir ce que dbt a réellement envoyé à DuckDB :

```
dbt compile --select fct_ventes
```

Ouvrez `target/compiled/boreal_ae/models/marts/fct_ventes.sql`. Deux choses y sautent aux yeux. Le modèle intermédiaire est devenu une expression de table commune nommée `__dbt__cte__int_lignes_enrichies`, collée en tête de la requête : voilà à quoi ressemble un `ephemeral` une fois compilé. Et l'appel de macro est devenu `md5(cast(coalesce(cast(code_concessionnaire as TEXT), '_dbt_utils_surrogate_key_null_') as TEXT))`, où se lit le traitement des valeurs vides dont la théorie parle.

C'est la commande à lancer chaque fois qu'un modèle fait quelque chose que vous ne comprenez pas.

Étape 9 — Le grain, puis la réconciliation

Théorie : `Theorie_04_AE.md`, section 8.

Le premier contrôle est celui du grain, et il se fait avant tous les autres :

```
select
    count(*) as lignes,
```


© 2026 iAlexMG. Tous droits réservés.

```
    count(distinct (numero_commande, numero_poste)) as couples_distincts
from marts.fct_ventes;
```

Attendu : 75 652 et 75 652. Si les deux nombres diffèrent, arrêtez tout : une jointure a dupliqué des lignes, et aucun total ne veut plus rien dire.

Le second est celui des totaux :

```
select
    count(*) as lignes,
    count(distinct numero_commande) as commandes,
    sum(valeur_nette) as valeur_nette,
    sum(cout) as cout,
    sum(marge) as marge
from marts.fct_ventes;
```

Mesure	Attendu
Lignes	75 652
Commandes	25 256
Valeur nette	430 148 012,16 $
Coût	312 784 347,46 $
Marge	117 363 664,70 $

Le troisième est la réconciliation, et c'est celui qui décide si l'étoile se livre. L'ERP porte sur chaque en-tête une valeur nette qu'il a calculée lui-même à partir de ses postes non refusés. Votre fait part des postes ; le témoin part des en-têtes.

```
select
    (select sum(valeur_nette) from marts.fct_ventes) as fait,
    (select sum(valeur_nette) from staging.stg_commandes) as erp_entetes,
    (select sum(valeur_nette) from marts.fct_ventes)
        - (select sum(valeur_nette) from staging.stg_commandes) as ecart;
```

Attendu : 430 148 012,16 $, 430 148 012,16 $, et un écart de 0,00 $. Aucune tolérance ici.

Deux écarts restent, et ils s'expliquent plutôt qu'ils ne se corrigent. Les 75 652 lignes ne sont pas les 77 732 postes de l'étape 2 : 2 080 sont refusés. Les 25 256 commandes ne sont pas les 25 483 de l'ERP : 227 commandes n'ont que des postes refusés et disparaissent entièrement du fait.

Reste à vérifier que l'étoile répond comme le prototype. Reposez la question du métier du Labo 02, mot pour mot, en remplaçant `prototype` par `marts` :

```
select
    p.gamme,
```


© 2026 iAlexMG. Tous droits réservés.

```
    d.trimestre,
    sum(f.valeur_nette) as valeur_nette
from marts.fct_ventes f
join marts.dim_produit p on p.cle_produit = f.cle_produit
join marts.dim_date d on d.date_jour = f.cle_date
join marts.dim_concessionnaire c on c.cle_concessionnaire = f.cle_concessionnaire
where c.province = 'QC'
  and d.annee = 2025
  and c.palier = 'Or'
group by 1, 2
order by 1, 2;
```

Les motoneiges rendent 574 546,57 $, 28 525,10 $, 3 273 517,55 $ et 910 380,72 $ par trimestre, les mêmes chiffres qu'au Labo 02. La conception n'a pas bougé ; c'est le reste qui a changé.

Étape 10 — Déclarer ce qu'on livre

Théorie : `Theorie_04_AE.md`, section 8.

Créez `LIVRAISON.md` à la racine de `02_ae`. C'est le document que les formations en aval liront pour savoir ce qu'elles reçoivent, comme vous avez lu celui de Data Engineering au Cours 01.

Sa première section, que vous complèterez jusqu'au capstone :

```
# Livraison — ce qu'Analytics Engineering livre aux formations suivantes

Commencé au Cours 04, complété cours après cours. Les chiffres sont ceux de la génération
par défaut de Boréal, à l'état du 2025-12-31.

## 1. L'étoile des ventes

Processus : un concessionnaire commande. Grain : une ligne de commande, hors postes refusés.
Où : schéma `marts` de `Projet_Boreal/plateforme/entrepot/boreal.duckdb`.
Reconstruction : `dbt build` depuis `Projet_Boreal/02_ae`.

| Modèle | Grain | Lignes | Fabriqué au |
|---|---|---|---|
| dim_date | un jour | 1 551 | Cours 04 |
| dim_concessionnaire | un concessionnaire, type 1 | 156 | Cours 04 |
| dim_produit | un article | 68 | Cours 04 |
```


© 2026 iAlexMG. Tous droits réservés.

```
| fct_ventes | une ligne de commande | 75 652 | Cours 04 |

Contrôle de livraison : la somme de fct_ventes.valeur_nette vaut 430 148 012,16 $, soit
exactement la valeur nette des en-têtes de commande de l'ERP.

Ce que cette livraison ne porte pas encore : l'historique des paliers. dim_concessionnaire
est de type 1 et attribue à toutes les ventes le palier d'aujourd'hui. Le Cours 05 corrige.
```

La dernière ligne est la plus importante des dix. Une livraison qui ne dit que ce qu'elle contient laisse le lecteur découvrir seul ce qu'elle ne contient pas, et il le découvrira dans un rapport, un mercredi après-midi.

Récapitulatif

L'état du projet à la fin du labo :

Ce qui est en place	**Vérification**
Le paquet	`dbt deps` installe `dbt_utils` 1.4.1, `dbt_packages/` reste hors du dépôt
Quatre sources déclarées	`erp_vbap` s'ajoute aux trois du Cours 03
Cinq vues de staging	`stg_postes` s'ajoute aux quatre du Cours 03, 77 732 postes
Un modèle intermédiaire	présent dans `dbt ls`, absent de la base
Quatre tables dans `marts`	`dbt run` rend `PASS=9 WARN=0 ERROR=0`
La dimension des dates	1 551 jours, libellés en français, 0 nul
Les deux dimensions tirées du staging	156 concessionnaires, 68 articles, clés uniques
Le fait	75 652 lignes, autant de couples distincts
La réconciliation	0,00 $ d'écart avec les en-têtes de l'ERP
La livraison	la première section de `LIVRAISON.md`

Le Cours 05 reprend cette étoile pour prouver qu'elle dit vrai : les tests, le snapshot qui rend au concessionnaire son palier d'époque, et la documentation que d'autres liront.

Pensez à commiter :

© 2026 iAlexMG. Tous droits réservés.

```
git add packages.yml package-lock.yml dbt_project.yml models LIVRAISON.md
git commit -m "Cours 04 : l'etoile des ventes"
```

Dépannage

Le message ou le symptôme	Ce qu'il veut dire	La parade
`dbt expects 1 package(s) ... Run "dbt deps" to install package dependencies`	`packages.yml` déclare un paquet qui n'est pas installé	lancer `dbt deps`
`FileNotFoundError` pendant `dbt deps`, sur un fichier du paquet	le chemin du projet est trop long pour Windows, qui s'arrête à 260 caractères	ranger `Projet_Boreal` à un chemin court ; `subst` ne contourne rien
`Runtime Error in model fct_ventes`, sur une colonne que `fct_ventes` ne nomme pas	l'erreur vient du modèle `ephemeral` qu'il appelle	ouvrir le SQL compilé, dont le message donne le chemin, et y retrouver la ligne
`Catalog Error: Table with name "intermediate.int_lignes_enrichies" does not exist because schema "intermediate" does not exist`	un modèle `ephemeral` ne crée aucun objet	l'interroger à travers le modèle qui l'appelle, ou lire son SQL compilé
`dbt run --select int_lignes_enrichies` ne construit rien et ne dit rien	même raison	sélectionner un modèle en aval, ou lancer `dbt run`
Les tables atterrissent dans `staging_marts`	la macro `generate_schema_name` manque	reprendre l'étape 3 du Labo 03
Le fait porte plus de lignes que de couples distincts	une jointure a dupliqué des lignes	vérifier que chaque jointure tombe sur une seule ligne de l'autre côté
La valeur nette vaut 448 149 563,37 $	les postes refusés n'ont pas été écartés	le filtre `where motif_refus is null` de l'étape 7
`nom_jour` est nul un jour sur sept, sans erreur	`dayofweek()` a été employé plutôt qu'`isodow()` : il rend 0 le dimanche, et une liste DuckDB s'indexe à partir de 1	`isodow()`, qui rend 1 pour lundi et 7 pour dimanche

© 2026 iAlexMG. Tous droits réservés.